1 Linux Course

1.1 For AI & Robotics Applications (Raspberry Pi + ROS)

This course is a practical Linux foundation for AI and robotics learners. It is designed for development on Linux machines and VMs, with Raspberry Pi concepts integrated across sessions.

GitHub repository:

https://github.com/KhaledMahfouz5/linux-course

Course structure: 3 chapters, 11 sessions
Session duration: 60-90 minutes each
Hardware note: Physical Raspberry Pi is optional

1.2 Requirements

- Basic programming knowledge (variables, conditions, loops, functions)
- Internet connection for searching the web and installing packages (most of the time you do not need a huge bandwidth)
- 30-50 GB free storage
- A note-taking method during class

1.3 Chapter 1: Introduction, Installation, and Embedded Linux Basics

1.3.1 Session 1: Linux for Robotics & AI

> **Before class:** Watch the linked videos for context. Come curious.

1.3.1.1 Learning outcomes By the end of this session, students should be able to: - Explain why Linux dominates robotics and AI workflows - Distinguish between the kernel and a full operating system - Choose suitable Linux distributions for robotics learning and deployment - Understand the Unix philosophy and free software movement - Evaluate Linux strengths/limitations for embedded and AI use-cases - Understand the Raspberry Pi ecosystem at a conceptual level

---

1.3.1.2 1) Why Linux Matters Today If you look at the modern world—servers, cloud platforms, supercomputers, phones, routers, IoT, cybersecurity labs, programming environments—Linux is *everywhere*. It's the invisible engine running most of the technologies you interact with daily.

A few facts: - **100%** of the world's top 500 supercomputers run Linux - **Most web servers** use Linux (Apache, Nginx) - **Android**, the most widely-used mobile OS, is built on the Linux kernel - **Cybersecurity distributions** (Kali, Parrot) are Linux-based - Developers choose Linux for transparency, speed, customizability, and control

So even if you've never installed Linux on your laptop, you've definitely used it indirectly.

For robotics and AI, this matters because the same OS family runs: - Development laptops - Edge devices (Pi/Jetson-like boards) - CI/CD pipelines - Data and model-serving infrastructure

Linux gives one consistent operational model from prototype to deployment.

1.3.1.3 2) Understanding Linux: What It Really Is One of the biggest misconceptions is thinking Linux is "that thing that replaces Windows." Linux is not a program. It's not a GUI. It's not even an operating system by itself.

Linux = the kernel

The kernel is the core program that: - Talks to hardware - Manages memory - Schedules processes - Handles permissions - Controls I/O

Everything else—the desktop, apps, package manager, and utilities—comes from the broader free-software ecosystem called **GNU**.

Together:
GNU tools + Linux kernel = GNU/Linux (a complete OS)

This separation is important because Linux teaches you something deeper than using an OS—it teaches you *how systems work*.

1.3.1.4 3) Kernel vs OS (embedded perspective) **Kernel** = low-level core that manages CPU, memory, processes, devices, and system calls.
Operating system = kernel + userland tools + services + package system + optional desktop.

Embedded systems usually optimize the userland while relying on the same Linux kernel principles: - Hardware abstraction through drivers - Process isolation - Permission model - Filesystem-based device interfaces

---

1.3.1.5 4) The Unix Philosophy Linux inherits its design principles from Unix. This philosophy will shape how you think during the whole course:

1. **Write programs that do one thing, and do it well**
2. **Write programs to work together**
3. **Write programs to handle text, because text is a universal interface**

This leads to: - Tools that are small and powerful - Commands that do simple tasks - The magic happens when you combine them using pipes (|)

It's why a Linux power-user is unstoppable—every small tool becomes a Lego block, and you can build anything from those blocks.

1.3.1.6 5) Linux History (The Simple Version) Let's keep it practical.

- **1969** – Unix born. AT&T created Unix. Clean, elegant, powerful.
- **1983** – GNU Project launched. Richard Stallman began building a free Unix-like system. GNU had everything—compilers, libraries, tools—*except a kernel*.
- **1991** – Linus Torvalds writes the Linux Kernel. As a personal hobby project. He released it under a free license, allowing anyone to study, modify, and contribute.
- **1992** – GNU + Linux merged. This created the first fully free operating system.
- **1990s–2000s** – Linux takes over servers. Why? Stability, control, security, no licensing fees.
- **2010s–2020s** – Linux becomes everywhere. Cloud, containers, mobile, embedded systems, DevOps… The world quietly standardized on Linux.

That's why learning Linux today is a **career skill**, not just a personal preference.

1.3.1.7 6) The Free Software Philosophy (Why Linux Exists) Linux exists because of the free software movement. Free software is about *freedom*, not price.

The Four Essential Freedoms: 1. **Freedom 0:** Run the program for any purpose 2. **Freedom 1:** Study the source code and modify it 3. **Freedom 2:** Distribute copies 4. **Freedom 3:** Distribute modified versions

If a program denies any of these freedoms, it becomes **proprietary**, which means: - You can't see how it works - You can't fix it - You depend on the vendor - You lose control over your own computer

Linux was built to solve this. During this course, you'll see the impact of this philosophy: - Software you can fully audit - Tools you can modify - Community-built contributions - Transparency at every layer

This is why cybersecurity professionals prefer Linux .. you can see *everything* happening in the system.

1.3.1.8 7) Why Robotics Engineers Choose Linux

> → **See Session 6 §4-§5** for deep dive into permissions, root access, and safe privilege usage.

- Strong CLI tooling for automation and debugging
- Better compatibility with robotics stacks (ROS ecosystem)
- Native package and build tooling for C/C++/Python workflows
- Reliable remote management over SSH
- Easy scripting for repetitive hardware/software tasks
- Everything is a file—devices, sockets, processes, IPC… all represented as files. Simplifies development massively
- Powerful terminal environment—you can automate anything
- Package managers—install complete development stacks in seconds
- DevOps and cloud-native tools—Docker, Kubernetes, CI pipelines—all built on Linux
- High performance and stability—perfect for servers and embedded systems
- Customizability—modify your environment to match exactly how you think and work

---

1.3.1.9 8) Linux Distributions for Robotics

If Linux is the engine, a **distro** is the full car.

Ubuntu: most common ROS learning path, huge documentation, easy onboarding
Debian: very stable and lightweight, strong base for long-term systems
Raspberry Pi OS (Raspbian): Pi-focused integration and educational accessibility

Selection criteria: - Stability and long-term support - Package availability - Hardware support - Team familiarity

Beginner-friendly: Ubuntu, Linux Mint, MX Linux
Power-user: Arch Linux, Gentoo, Void Linux

Enterprise: Red Hat Enterprise Linux, SUSE Linux Enterprise
Security: Kali Linux, Parrot OS

> **Recommendation:** For this course, use desktop Ubuntu or Debian for easiest package support and lab consistency. Avoid Arch-based systems as your first Linux unless you're ready for deeper learning.

1.3.1.10 9) Pros and Cons of Linux in Robotics/AI **Pros:** - High control and transparency - Excellent automation capabilities - Strong networking and remote tooling - Efficient on modest hardware - Powerful CLI - Free and open source - Fast, lightweight - Secure and stable - Full control over your system - Great for programming and networking - Massive community support

Cons: - Higher learning curve for beginners - Some device/vendor tooling may be Windows-first - Some hardware drivers might be missing - Gaming is improving but still imperfect - Requires operational discipline (permissions, services, system config) - Requires learning mindset, not point-and-click

But as a developer, engineer, or cybersecurity student, the *pros* outweigh the cons by a huge margin.

1.3.1.11 10) Raspberry Pi Ecosystem Overview (Conceptual) Key idea: Raspberry Pi is a compact Linux computer often used for sensing, control, and edge inference.

Ecosystem concepts: - GPIO for digital I/O - Camera and CSI-based vision capture - I2C/SPI/UART buses for sensors/actuators - SD-card based storage and boot - SSH-first/headless management patterns

---

1.3.1.12 11) How to Think Like a Linux User To succeed in this course:

Adopt these habits: - **Be curious** — try commands, explore folders - **Read error messages** — Linux errors are usually helpful - **Use man pages** — every command has documentation - **Experiment safely** — break things in virtual machines - **Think in small tools** — pipes, filters, text processing - **Love the terminal** — it's your real power

Linux rewards people who explore.

1.3.1.13 Useful Links — Session 1 Video Introductions:

- Linux History & Distros
- Unix Philosophy
- Kernel vs OS
- Free Software Philosophy
- Why Programmers Love Linux
- Pros and Cons

Recommended Arabic YouTube Channels:

- anaHr
- mohammad besar
- Al-Waqqad
- sudostart
- linuxtopia
- Abdulmojeeb Al-Hameed

Book: *The Art of UNIX Programming* — Eric Raymond

1.3.2 Session 2: Installation & Embedded Setup

> **Before class:** Install VirtualBox/VMware. Optional: QEMU for Pi emulation.

1.3.2.1 Learning outcomes

- Install Linux via VM or dual boot safely
- Understand headless setup basics (SSH/WiFi/static IP)
- Perform post-install essentials for reliable daily use
- Explain the role of Camera/I2C/SPI/UART in Pi-based systems

---

1.3.2.2 1) Choosing Your Linux Distribution

> → **See Session 1 §8** for detailed distro categories (beginner, power-user, enterprise, security) and selection criteria.

Before installing Linux, you must pick the right **distro** (distribution). Think of distros like different flavors of the same operating system—they all share the Linux kernel but differ in user interface, package managers, performance, and philosophy.

What You Should Look For:

Requirement	What it means	Good Choices
Ease of use	Beginner-friendly UI, simple updates	Ubuntu, Linux Mint, Pop!_OS
Stability	Fewer issues & long-term support	Ubuntu LTS, Debian
Performance	Works well on older hardware	Xubuntu, Linux Lite
Security	Good by default, strong community	Fedora, Ubuntu
Software availability	Access to apps & drivers	Ubuntu, Fedora

Quick Tips: - If your PC is older than **10 years**, pick **Xubuntu**, **Linux Mint XFCE**, **LMDE**, **MX-Linux**, or **Zorin OS** - If you're into **gaming**, choose **Pop!_OS** or **Nobara OS** - If you want a macOS-like UI, choose **elementaryOS**

1.3.2.3 2) Primary Setup: Full Linux Installation (Dual Boot or VM) Before You Install – VERY Important:

A. Backup Your Files
The installer will delete the Windows partition if you're switching fully. Backup all files from Desktop, Documents, Pictures, Music, Videos. Move them to D: (or another partition), external HDD, or USB stick.

B. Check Your Boot Mode
Open Start → type **System Information** → look for **UEFI** (modern, secure boot) or **Legacy BIOS** (older systems). This affects partitioning and bootloader behavior.

C. Create a Bootable USB

Recommended tool: **Ventoy**—supports multiple ISOs on the same flash drive, very beginner-friendly.

Try Ubuntu Before Installing:

Boot from the USB and choose “Try Ubuntu” (Live Mode). Test Wi-Fi, Bluetooth, sound, display, keyboard, touchpad. If everything works → safe to install.

VM Path (Recommended for Beginners): - Low risk, fast reset, easy snapshots - Good for command practice and repeatable labs

Recommended VirtualBox Settings: - EFI: Choose What Gives You The Best Experience . - CPU: 2-4 cpu cores . - Display: 128 MB VRAM - Network: NAT (Easy For Beginners) - RAM: 4-6 GB allocated

Installing Guest Additions:

```
sudo apt install build-essential dkms linux-headers-$(uname -r)
sudo sh /media/username/VBox*/VBoxLinuxAdditions.run
```

This enables auto-resize display, clipboard sharing, drag & drop, and better performance.

Dual-Boot Path (Optional): - Better native performance - Requires careful partitioning and backup discipline - Choose “Something Else (Manual Partitioning)” .. DO NOT choose “Erase disk” or “Install alongside Windows”

Recommended Partition Layout:

Partition	Size	Filesystem	Mountpoint	Notes
EFI/System	512–1024 MB	FAT32	`/boot/efi`	Needed only for UEFI
ROOT	40–100 GB+	EXT4	/	Main system
SWAP	Optional (2–4 GB)	swap	swap	Only if RAM < 8GB
Data	Keep	NTFS	`/mnt/data1`	Do *not* format

> **Beginner’s Tip:** Don’t create a separate `/home` partition. You can add it later when you’re more experienced.

1.3.2.4 3) Headless Setup Concepts

Headless means “manage without monitor/keyboard on target device.”

Core concepts: - SSH service enabled on boot - Network reachability (WiFi/Ethernet) - Static IP plan for stable robot addressing

---

1.3.2.5 4) Post-Install Essentials

Update System:

```
sudo apt update && sudo apt upgrade -y
```

Install Ubuntu Restricted Extras (codecs for MP3, MP4, MOV, Fonts):

```
sudo apt install ubuntu-restricted-extras
```

Add Flatpak + Flathub:

```
sudo apt install flatpak
flatpak remote-add --if-not-exists flathub https://flathub.org/repo/flathub.flatpakrepo
sudo apt install gnome-software gnome-software-plugin-flatpak
```

AppImage Support:

```
sudo apt install libfuse2
```

GNOME Extensions & Tweaks:

```
sudo apt install chrome-gnome-shell gnome-shell-extension-manager gnome-tweaks
```

Minimize on Click:

```
gsettings set org.gnome.shell.extensions.dash-to-dock click-action 'focus-minimize-or-appspread'
```

Enable New Document Menu:

```
touch ~/Templates/new
```

System Backups: - **Timeshift** (System Snapshots): `sudo apt install timeshift` - **Déjà Dup** (User files backup): Built into Ubuntu

Essential Security: - Install drivers and core tooling - Enable SSH server - Configure firewall baseline - Create backup/snapshot routine

Final System Update:

```
sudo apt update && sudo apt upgrade -y && flatpak update -y && sudo
    snap refresh
```

1.3.2.6 5) Pi Interfaces (Theoretical)

> → **See Session 1 §10** for the full Raspberry Pi ecosystem overview (GPIO, Camera, I2C, SPI, UART, SD-card, SSH patterns).

- **Camera:** image/video input for CV pipelines
- **I2C:** low-speed bus for many sensors
- **SPI:** higher-speed peripheral communication
- **UART:** serial communication for diagnostics/controllers

1.3.2.7 Practice

Run these commands to verify your SSH and firewall setup:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y openssh-server ufw
sudo systemctl enable --now ssh
sudo ufw allow OpenSSH
sudo ufw enable
ip a
```

1.3.2.8 Useful Links — Session 2 Video Tutorials:

🔗 Choosing Your Distro

🔗 Install Linux Step-by-Step

🔗 Dual-Boot with Windows

🔗 VirtualBox VM Setup

Written Guides:

🔗 Ubuntu 24.04 Post-Install Guide

1.3.3 Session 3: Linux Basics for Robotics

1.3.3.1 Learning outcomes

- Navigate Linux confidently
- Use core command-line tools for daily robotics development
- Understand path usage for ROS workspaces
- Explore device/hardware-facing paths (`/dev`, `/sys`)

1.3.3.2 1) Introduction to the Linux Command Line The Linux command line (often called the **terminal**, **shell**, or **CLI**) is a text-based interface that allows you to interact with your system quickly and efficiently. While graphical interfaces (GUIs) are great for daily tasks, the command line provides:

- Faster automation
- Power-user flexibility
- Access to advanced tools
- A universal interface across all distros

Most distributions ship with **bash** as the default shell, though others (zsh, fish) also exist.

To open a terminal: - **Ubuntu / Debian**: `Ctrl + Alt + T` - **Fedora**: Activities → Terminal - **Any distro**: Search “Terminal”

1.3.3.3 2) System Information and Hardware Detection Linux provides several commands to check system details:

```
uname -a        # All system info
uname -r        # Kernel version
uname -m        # Architecture
hostnamectl     # OS name, kernel, hardware
```

inxi (advanced—may need installation):

```
sudo apt update
sudo apt install inxi
inxi -F         # CPU, GPU, RAM, kernel, drivers
```

lsb_release (Debian-based):

```
lsb_release -a
```

1.3.3.4 3) Shell Variables and Environment Configuration

The shell stores data (variables) temporarily during your session.

```
echo $HOME       # Home directory
echo $USER       # Current user
echo $PATH       # Executable search paths

NAME="Khaled"    # Create variable
echo $NAME

export EDITOR=nano    # Export (available to programs)

printenv         # List all variables
```

Environment variables define runtime behavior for tools and scripts.

1.3.3.5 4) File System Navigation

Every Linux filesystem starts from the **root (/)** directory.

```
pwd              # Print working directory
ls               # List files
ls -l            # Long format
ls -la           # Include hidden files
```

cd – Change Directory:

```
cd /home         # Absolute path
cd Documents     # Relative path
cd ..            # Parent directory
cd ~             # Home directory
cd -             # Previous directory
```

Shortcuts: - **.** → current directory - **..** → parent directory - ~ → home directory - − → previous directory

---

1.3.3.6 5) Essential Commands

`ls`, `cd`, `pwd`, `cat`, `mkdir`, `cp`, `mv`, `rm`, `echo`, `uname`, `inxi` are the baseline daily toolkit.

```
mkdir -p robotics_ws/src
cd robotics_ws
pwd
echo "session3" > notes.txt
cat notes.txt
cp notes.txt notes.bak
mv notes.bak archive.txt
touch newfile.txt
sleep 5
```

rm – Remove (⚠ Dangerous but essential):

```
rm file.txt
rm -r folder/
rm -rf folder/    # VERY DANGEROUS - use with care!
```

1.3.3.7 6) Paths: Relative vs Absolute

Robotics workspaces fail often because of path mistakes.

Absolute Paths start from the root `/`:

```
/home/khaled/projects/code.py
```

Relative Paths start from your current location:

```
../images/photo.png
./run.sh
```

- **Absolute:** starts from `/` (stable in scripts)
- **Relative:** from current directory (fast interactive use)

1.3.3.8 7) Package Managers: `apt`, `pip`, `snap`

apt – Debian, Ubuntu, Linux Mint:

```
sudo apt update             # Update repositories
sudo apt install gedit      # Install package
sudo apt remove gedit       # Remove package
sudo apt upgrade            # Upgrade system
```

dnf – Fedora, RHEL 8+:

```
sudo dnf install nano
sudo dnf remove nano
sudo dnf update
```

yum – Older RHEL/CentOS:

```
sudo yum install nano
sudo yum update
```

- `apt` for system packages
- `pip` for Python packages
- `snap` for bundled apps/tooling where appropriate

1.3.3.9 8) Editors: `nano`, `gedit`, `vim` Basics **nano (Terminal Editor):**

```
nano file.txt
```

Controls: - Save: **Ctrl + S** - Exit: **Ctrl + X** - Search: **Ctrl + W**

gedit (GUI Editor):

```
gedit notes.txt &    # & lets the terminal continue
```

Students should be comfortable editing files both in terminal and GUI contexts.

1.3.3.10 9) Help Tools

> → **See Session 1 §11** for the mindset behind using documentation and self-learning in Linux.

Linux has multiple built-in help resources.

```
man ls              # Manual pages
man tar
info coreutils      # Info documentation
tldr ls             # Simplified examples (install first)
ls --help           # Quick help
apropos network     # Search commands by description
```

Use `man`, `info`, `tldr`, `--help`, and `apropos` to self-debug and self-learn.

1.3.3.11 10) Robotics Context: `/dev/` and `/sys/`

In Linux: - Files - Directories - Devices - Hardware - Network interfaces

→ **Everything is a file**

`/dev/` (device files):

```
/dev/sda          # Hard drive
/dev/null         # Black hole (silence output)
/dev/tty          # Current terminal
/dev/random       # Random number generator
```

`/sys/` and `/proc/` (kernel-exposed hardware/state metadata):

```
/proc/cpuinfo          # CPU info
/proc/uptime           # System uptime
/proc/net/tcp          # TCP connections
/sys/class/net/eth0/address           # Network MAC address
/sys/class/backlight/brightness       # Screen brightness
/sys/class/power_supply/BAT0/capacity # Battery level
```

Want hardware info? `cat /proc/cpuinfo`
Want to silence output? `command > /dev/null`

This design is WHY Linux is scriptable, automatable, and runs the internet.

1.3.3.12 11) Explore Virtual Hardware Seen by VM

Inspect which virtual disks, network interfaces, and pseudo-devices appear.

1.3.3.13 12) Archiving: `tar`, `zip`

Archive projects and logs for sharing, reproducibility, and backups.

tar (most common):

```
tar -cvf archive.tar folder/           # Create
tar -xvf archive.tar                   # Extract
tar -czvf archive.tar.gz folder/       # Compressed (gzip)
tar -xzvf archive.tar.gz               # Extract compressed
```

zip:

```
zip -r robotics_ws.zip robotics_ws
unzip archive.zip
```

7z (7-Zip):

```
sudo apt install p7zip-full
7z a archive.7z folder/
7z x archive.7z
```

1.3.3.14 Bonus Useful Commands

- `clear` – clears the terminal
- `history` – shows previous commands
- `head` / `tail` – preview files (`tail -f log.txt` for live logs)
- `less` – view files with navigation
- `grep` – search text
- `whoami` – current user
- `df -h` – disk usage
- `du -sh folder/` – folder size

1.3.3.15 Practice Exercises

1. Create a directory structure: `projects/python/basics`
2. Create three files inside `basics`
3. Write your name into `info.txt` using `echo`
4. Compress the folder using tar and zip
5. Install `tldr` and test 5 commands
6. View your system kernel version and CPU info using two methods

---

1.4 Chapter 2: Programming, Development Tools & ROS Foundation

1.4.1 Session 4: Text Manipulation & Log Analysis

> Welcome to one of the most *useful and exciting* sessions in your Linux journey. In this session, we learn how Linux becomes a **superpower** when working with text, logs, automation, scripting,

> and filtering.

1.4.1.1 Learning outcomes

- Filter and transform large text/log outputs efficiently
- Extract structured fields from sensor logs
- Locate files quickly in large ROS-style directories
- Chain commands with pipes/redirection

1.4.1.2 1) `grep` — The Search Engine of the Terminal

`grep` stands for **Global Regular Expression Print**. In simple words: **grep searches for text inside files or command output.**

```
grep "error" /var/log/syslog                    # Find lines with "error"
grep -i "linux" notes.txt                       # Ignore case
grep -r "password" /etc                         # Recursive search
grep -n "TODO" project.py                       # Show line numbers
grep -v "DEBUG" logs.txt                        # Invert match (NOT
    containing)
grep -E "cat|dog" animals.txt                   # Extended regex (OR)
grep -i "error" robot.log
grep -nE "warn|fail|timeout" robot.log
```

Practical examples:

```
ls | grep ".pdf"                                # Find PDF files
grep "failed" *.log                             # Search in multiple logs
grep -E "[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+" nginx.log  # Find IP addresses
```

1.4.1.3 2) `sed` — The Stream Editor

`sed` is a **text transformer**. You can use it to replace words, delete lines, insert text, extract text, and edit files *without opening them*.

```
sed 's/old/new/' file                           # Replace first occurrence
    per line
sed 's/old/new/g' file                          # Replace ALL occurrences
sed 's/linux/Linux/' notes.txt
sed '/error/d' logs.txt                         # Delete lines with "error"
sed -n '/INFO/p' logs.txt                      # Print only matching lines
sed -i 's/foo/bar/g' file.txt                  # \faIcon{exclamation-
    triangle} Edit IN-PLACE (BE CAREFUL!)
```

Cool examples:

```
sed 's/ //g' text.txt                          # Remove all spaces
sed 's/  */ /g'                                # Multiple spaces to one
sed 's/^/[INFO] /' file.txt                   # Add prefix to each line
sed 's/WARN/WARNING/g' robot.log > robot_clean.log
sed -n '/camera/p' robot.log
```

1.4.1.4 3) awk — The Text Processor King

awk is a **programming language** hidden inside a terminal command. Perfect for extracting columns, summing numbers, processing CSV files, and analyzing logs.

```
awk '{print $1}' file.txt                      # Print first column
awk '{print $2}' data.txt                      # Print second column
awk '$2 > 50' data.txt                         # Lines where 2nd field >
    50
awk '{sum += $3} END {print sum}' sales.txt # Sum column 3
awk -F',' '{print $1, $3}' sensor.csv       # CSV with comma delimiter
awk '$2 > 50 {print $0}' telemetry.txt
```

Example—given line `Alice 23 Student`: - `$1` = Alice - `$2` = 23 - `$3` = Student

---

1.4.1.5 4) `find` — The Ultimate Search Tool

`find` searches your filesystem **recursively** for files, directories, sizes, permissions, names, types—everything. Essential in nested workspaces and mixed code/config trees.

```
find . -name "*.txt"                           # Search by name
find /home -iname "linux"                      # Case-insensitive
find . -type d -name "src"                     # Directories only
find . -size +1M                               # By size
find / -perm 777                               # By permissions
find . ! -name "*.txt"                         # Exclude pattern
find ~/robotics_ws -name "*.launch"
find ~/robotics_ws -type f -name "*.py"
```

1.4.1.6 5) Pipes and Redirections

Pipes (|) send the output of one command into another. Think of it as plugging commands together like LEGO pieces.

```
ls | grep ".txt"                              # Filter listing
ls | grep ".py" | wc -l                       # Count Python files
du -ah | sort -h | tail                       # Top 10 largest files
cat text.txt | tr ' ' '\n' | sort | uniq    # Unique words
```

Redirections: - | passes output between commands - > overwrite output file - >> append output file - < feed file as input

```
echo "Hello" > file.txt                       # Write to file
echo "World" >> file.txt                      # Append to file
ls > list.txt                                 # Redirect output
wc -l < notes.txt                             # Input from file
```

1.4.1.7 6) Practical Robotics Application

Filter ROS-like outputs, isolate anomalies, and summarize key fields for debugging.

```
cat robot.log | grep "camera" | awk '{print $1,$2,$NF}'
grep -i "imu" topics.txt | sort | uniq -c
```

1.4.1.8 Useful Links — Session 4 Video Tutorials:

- grep
- sed
- awk
- find

1.4.2 Session 5: Development Environment for AI & Robotics

> **Verify installed:** `python3`, `python3-venv`, `git`, `cmake`, `build-essential`

1.4.2.1 Learning outcomes

- Set up isolated Python development environments
- Install and manage AI/vision packages
- Understand source-build basics
- Use team Git workflows
- Prepare editor + Docker-based development flow

1.4.2.2 1) Mindset & Setup Linux for software dev is about **three ideas**: 1. Everything is a tool 2. The terminal is your superpower 3. You *own* your system

Update first. Always:

```
sudo apt update && sudo apt upgrade -y
```

Install the essentials:

```
sudo apt install -y build-essential curl wget git software-properties-
    common
```

1.4.2.3 2) Python Setup (`venv`, `pip`) Use virtual environments to avoid global dependency conflicts. **DO THIS.**

```
sudo apt install -y python3 python3-pip python3-venv
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install requests bs4
```

1.4.2.4 3) AI/ML Framework Installation Start with OpenCV and expand to TensorFlow/PyTorch according to hardware and project scope.

```
pip install opencv-python
```

1.4.2.5 4) Compile from Source (Makefiles/Build Systems) Understand compile-link pipeline and why build systems are central in robotics C/C++ codebases.

C/C++ Compilers & Debugging:

```
sudo apt install -y gcc g++ gdb
gcc main.c -o main          # Compile
gdb ./main                  # Debug
```

Makefile basics:

```
all:
    gcc main.c -o main
```

```
make                        # Run Makefile
```

1.4.2.6 5) Git Workflow for Robotics Teams Branch, commit, push, and review with reproducible history and rollback capability.

1.4.2.7 6) Web Development & Servers **Simple HTTP server (classic):**

```
python3 -m http.server 8000
```

Now anyone on your network can access `http://YOUR_IP:8000`

> ⚠ Warning: `http.server` is not recommended for production. It only implements basic security checks.

Node.js:

```
# Install nvm (Node Version Manager)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh | bash
\. "$HOME/.nvm/nvm.sh"
nvm install 24

# Verify
node -v
npm -v
```

PHP:

```
sudo apt install -y php php-cli php-mysql
php -S localhost:8000
```

1.4.2.8 7) IDE Setup VS Code with Python/C++/ROS extensions for integrated linting, debugging, and navigation.

Install: - VS Code: - Browsers: `sudo apt install -y firefox` - Arduino IDE: `sudo apt install -y arduino`

1.4.2.9 8) Docker for Containerized Workflows Use containers for reproducible environments in model serving and simulation pipelines.

```
sudo apt install -y docker.io
sudo usermod -aG docker $USER
newgrp docker
docker run hello-world
docker run -it ubuntu bash
```

Distrobox (Run Any Distro Safely):

```
sudo apt install -y podman
distrobox create -n arch --image archlinux:latest
distrobox enter arch
```

Now you're in another distro. No VM. No pain.

1.4.2.10 9) Resource Optimization for Embedded Targets Account for CPU, RAM, thermal limits, and storage constraints before deployment.

1.4.2.11 10) Update System & Repositories

> → **See Session 2 §4** for the full post-install checklist including updates, codecs, Flatpak, Snap, and backups.

```
sudo nano /etc/apt/sources.list    # Edit sources
sudo apt update && sudo apt upgrade
```

> ⚠ **Kali Repositories Warning:** Don't mix Kali repos with Ubuntu casually. You *will* break things. Safe use case: **specific tools only**. Use pinning if you *must*.

1.4.3 Session 6: System Configuration & Tools for Robotics

1.4.3.1 Learning outcomes

- Navigate Linux filesystem with robotics context
- Manage permissions for hardware/device access
- Configure shell environment and productivity shortcuts
- Understand links and safe privilege usage

1.4.3.2 1) Filesystem Hierarchy in Robotics Context

> → **See Session 3** for hands-on navigation practice (`cd`, `ls`, `pwd`, paths). This section focuses on the *purpose* of each directory.

Linux doesn't use drive letters like Windows (`C:\`, `D:\`). Instead, **everything starts at one place**: `/` (the root).

`/` — The Root of Everything
`/` is the top of the file system tree. You **do not casually mess around in `/`**.

`/home` — Where Humans Live
This is where **users live**. Inside your home directory: Documents, Downloads, Config files, SSH keys, Hidden dot files. > • **Best Practice:** If you're learning Linux, live in `/home`

`/etc` — Configuration Central
Contains system configuration files, user settings, service configs, network configs.

```
/etc/passwd
/etc/shadow
/etc/hosts
/etc/sudoers
```

⚠ Editing files here can break login, networking, or lock you out completely.
> • **Rule:** `/etc` = configuration, not programs

`/var` — Variable Data (Stuff That Changes)
Logs, caches, spools, databases.

```
/var/log
/var/log/syslog
/var/log/auth.log
```

If a system is acting weird → **Check `/var/log` or related logs**

`/usr` — User Programs (Not Users)
Confusing name—this is **not user home directories**. Contains installed programs, libraries, shared resources.

```
/usr/bin
/usr/lib
/usr/share
```

`/dev/` — Hardware/Device Nodes (serial, camera, input)
`/sys/class/gpio/` — GPIO representation (conceptual Pi mapping)
`/opt/ros/` — ROS installation location

1.4.3.3 2) "Everything Is a File" (Linux Philosophy)

> → **See Session 3 §10** for the full explanation with `/dev/`, `/proc/`, and `/sys/` examples.

This is where Linux becomes *cool*. In Linux, files, directories, devices, hardware, and network interfaces are all represented as files.

```
/dev/sda           # Hard drive
/dev/null          # Black hole
/proc/cpuinfo      # CPU info
/dev/tty           # Current terminal
/dev/random        # Random number generator
/dev/urandom       # Faster random numbers
/proc/uptime       # System uptime
/proc/net/tcp      # TCP connections
/sys/class/net/eth0/address          # Network MAC
/sys/class/backlight/brightness      # Screen brightness
/sys/class/power_supply/BAT0/capacity # Battery level
/dev/loop0         # Loopback device (mount files as disks)
```

This design is WHY Linux is scriptable, automatable, and runs the internet.

1.4.3.4 3) Dotfiles (Hidden Power)

> → **See Session 3 §12** where `ls -la` first introduces hidden files. This section goes deeper.

Files that start with a dot `.` are **hidden**.

```
.bashrc
.profile
.gitconfig
.ssh/
```

They usually store shell configs, app preferences, and user settings. List them: `ls -a`

Why hide them? Prevent clutter and accidental deletion.
> • **Important:** Deleting dot files can reset your environment or break tools.

Dotfiles (`.bashrc`, `.profile`): Used to customize shell behavior, aliases, environment variables, and startup logic.

1.4.3.5 4) Permissions and Groups for Hardware Access **Why This Matters (Seriously):**

Linux doesn't break because it's fragile. Linux breaks because **permissions exist** and **you don't understand them yet**.

Permissions protect your system, prevent malware from wrecking everything, and make Linux powerful for servers, DevOps, security, and cloud. Once this clicks: errors make sense, "Permission denied" stops ruining your day, and you stop nuking your system with `sudo rm -rf`.

Every file has: - **Owner** - **Group** - **Permissions**

Check with:

```
ls -l
```

Example:

```
-rwxr-xr-- 1 ahmad staff script.sh
```

Breakdown: - Owner: ahmad (rwx) - Group: staff (r-x) - Others: (r–)

Three permission types: - **r** = read (4) - **w** = write (2) - **x** = execute (1)

Permission Numbers (Octal Mode):

```
7 = rwx
6 = rw-
5 = r-x
4 = r--
```

```
chmod 755 script.sh    # Owner: rwx, Group: r-x, Others: r-x
chmod +x script.sh     # Make executable
```

> • You will see 755 and 644 everywhere.

Changing Ownership:

```
chown user file
chown user:group file
sudo chown root:root config.conf
```

⚠ Wrong ownership = broken apps

Typical groups for hardware access: `dialout`, `gpio`, `video`. Correct group membership often resolves "permission denied" with serial/camera devices.

1.4.3.6 5) Root Permissions (`sudo`, `su`, `/etc/sudoers`)

Root is the system administrator—the power mode user. Root can read any file, delete any file, and change any permission.

`sudo` — Temporary Root:

```
sudo command
```

Executes command as root, logs actions, requires password.
> • **Best practice:** Use `sudo`, not permanent root login

`su` — Switch User:

```
su
su username
```

Switches shell to another user. ⚠ Dangerous if misused.

`/etc/sudoers` — Who Can Use sudo:
DO NOT edit directly. Always use:

```
sudo visudo
```

Wrong syntax here → no sudo → bad day.

Common Beginner Mistakes: - ❌Running everything with sudo (bad for security) - ❌`chmod 777` everything (everyone can do everything—bad for security & interviews :-)) - ❌Editing system files without backup (`sudo cp config.conf config.conf.bak`—always!) - ❌Deleting random stuff in `/etc` or `/usr` - ❌Not reading error messages (Linux tells you what failed and why)

> • Use sudo **only when required**. Permissions are not obstacles—they are **guardrails**.

1.4.3.7 6) Hard Links and Soft Links

Links are **references** to files.

Hard Links: - Point to the same inode - File exists in multiple locations - Deleting one doesn't remove data

```
ln file1 file2
```

Limitations: Same filesystem only, can't link directories.

Soft Links (Symlinks): - Pointer to file path - Like Windows shortcuts

```
ln -s target linkname
ln -s /var/log/syslog syslog_link
```

If target is deleted → ❌Link breaks
> • Most commonly used type.

1.4.3.8 7) Shell Aliases

> → **See Session 1 §11** for the mindset behind customization and efficiency.

Aliases save time—like macros for your shell. Speed up common workflows with carefully chosen aliases.

```
alias ll='ls -la'
alias gs='git status'
echo "alias ll='ls -lah'" >> ~/.bashrc
```

Add to `~/.bashrc` or `~/.zshrc` and `source ~/.bashrc` to reload. You'll thank yourself later when typing gets lazy :)

1.4.3.9 8) Font/Theme Customization

Long robotics sessions benefit from readable fonts and reduced visual fatigue.

- Install custom fonts: `sudo apt install fonts-roboto`
- GNOME Tweaks for themes
- Flatpak themes from Flathub

Fonts + themes = aesthetic ~100% more hipster points B-)

1.4.3.10 9) Virtual Device Exploration

Understand how simulated hardware appears in `/dev` and how tooling discovers it.

1.4.3.11 10) Install Common Software

> → **See Session 3 §7** for detailed package manager coverage (`apt`, `dnf`, `yum`, `pip`, `snap`).
> → **See Session 2 §4** for Flatpak and Snap setup details.

Quick starter pack for Ubuntu:

```
sudo apt update
sudo apt install \
    git htop tmux fastfetch
```

1.4.3.12 Useful Links — Session 6 Package Management:

- APT docs (Debian Wiki)
- APT Howto (Ubuntu Community)
- Debian SourcesList
- Ubuntu Repositories
- Kali Repositories docs
- Adding Kali repos to Debian (YouTube)

Build from Source:

- suckless st
- suckless dmenu
- GCC docs
- GDB docs
- GNU Make manual
- avr-libc

Web Development:

- Python http.server docs
- npm serve
- PHP Manual
- XAMPP download
- XAMPP FAQ (Linux)

Node.js & React:

- Node.js docs
- React docs

Java:

- OpenJDK
- Oracle Java Tutorial

Python:

- Python venv tutorial
- pip docs

Apps & IDEs:

- Google Chrome

VS Code

VS Code Linux Setup

Arduino

Containers:

Docker install docs

Docker tutorial (mmbesar)

Distrobox docs

AppImage:

AppImage docs

AppImage tutorial (mmbesar)

1.4.4 Session 7: Process Monitoring & Robotics System Management

> **Verify `htop` is installed.**

1.4.4.1 Learning outcomes

- Monitor process/resource usage under robotics workloads
- Control or terminate stalled tasks safely
- Use background execution and command chaining
- Reason about embedded constraints (thermal/memory throttling)

1.4.4.2 1) What Is a Process? Every program you run becomes a *process*—an instance of a running program with a unique ID (PID). Linux lets you **list**, **monitor**, and **control** these processes using simple tools.

1.4.4.3 2) `htop` — Interactive Process Viewer

`htop` is like `top`, but *way more human-friendly*: color, scrolling, arrow navigation, signal menus, and killer shortcuts.

Why `htop`? - Shows CPU, RAM, threads in real-time - Scroll through processes - Select and kill without typing PIDs

Install & Run:

```
sudo apt update
sudo apt install htop
htop
```

Inside `htop`: - ↑/↓ — move - F9 — kill selected process - F3 — search/filter - F5 — tree view (parent/child structure)

> Always use `htop` to **see what's happening right now**—you'll *instantly* understand resource bottlenecks.

1.4.4.4 3) `kill` and `pkill` — End Processes

When a process misbehaves (freezes, uses all your CPU), you kill it.

kill (targets by PID):

```
kill 1234               # SIGTERM - politely asks to quit
kill -9 1234            # SIGKILL - stops immediately
```

pkill (targets by name .. no need to look up PID):

```
pkill firefox
pkill -u someuser
```

[NOTE] Tip: `kill` works on PIDs, `pkill` works on *names*.
> ⚠ Use `kill` sparingly—force-killing can corrupt data if the program is mid-write.

1.4.4.5 4) `ps` + `grep` — Find Processes via CLI

`ps` gives you a *snapshot* of currently running processes—not real-time like `htop`, but extremely useful.

```
ps aux                  # List ALL processes
ps aux | grep -i ros    # Find ROS processes
ps aux | grep firefox   # Find Firefox
ps -ef | grep python    # Find Python instances
```

⚠ Pro tip: wrap your grep to *not* match itself (e.g., `grep "[f]irefox"`), because normal grep often shows its *own* process.

1.4.4.6 5) Background Execution (&) Sometimes you want to start a long-running task *without blocking your terminal*.

```
sleep 60 &                # Run in background
```

- & → run command in the background
- You get your prompt back immediately
- Shell assigns a job number and PID

Job Management:

```
jobs                      # List jobs
fg %1                     # Bring job 1 to foreground
bg %1                     # Resume job 1 in background
```

&, `screen`, `tmux` keep long tasks active while terminal remains usable.

1.4.4.7 6) Command Chaining — &&, ||, ; Command chaining lets you fire off multiple commands in one line—*with logic about success or failure*.

Operator	What it Means
`cmd1 ; cmd2`	Run cmd1, then cmd2 no matter what
`cmd1 && cmd2`	Run cmd2 ONLY if cmd1 succeeds (exit status 0)
`cmd1 || cmd2`	Run cmd2 ONLY if cmd1 fails

```
sudo apt update && sudo apt upgrade -y
rm temp.log || echo "Could not delete temp.log"
cmd1 ; cmd2 ; cmd3
```

• Perfect for chaining updates, scripts, and error handling. Practice chaining—it's a *game changer* for automation and scripting.

1.4.4.8 7) Embedded Constraints Understand temperature, throttling, and memory pressure behavior on small devices.

1.4.4.9 8) Real-Time Resource Management Prioritize critical processes and avoid overload from concurrent jobs.

1.4.4.10 9) Simulation Monitoring Track simulator CPU/RAM impact and tune settings to maintain responsiveness.

1.4.4.11 Useful Links — Session 7 **Process Management:**

- htop tutorial (GeeksforGeeks)
- Manage processes with ps, kill, pkill (How-To Geek)
- Viewing and Monitoring Processes (Ubuntu Community)

Job Control & Commands:

- Bash job control (DigitalOcean)
- Chaining Commands (GeeksforGeeks)

General References:

- Ubuntu CLI for Beginners
- Bash Reference Manual

1.5 Chapter 3: System Management, Scripting & Networking

1.5.1 Session 8: Automation with systemd & Cron

> Alright .. this is the session where you level up from "I run Linux" to: "I control what runs on Linux… and when it runs."

1.5.1.1 Learning outcomes

- Create/manage systemd services for robotics startup
- Schedule recurring tasks using timers and cron
- Understand unit dependencies and debugging basics

1.5.1.2 1) What is systemd? (And Why You Should Care) On modern Debian-based systems (Debian, Ubuntu, Linux Mint), the init system is **systemd**.

An *init system* is the first userspace process started by the kernel. Check it:

```
ps -p 1 -o comm=    # Should show "systemd"
```

What Does systemd Do? - Starts services during boot - Manages background processes (daemons) - Handles logging (via `journald`) - Mounts filesystems - Manages network services - Handles timers (our main topic today)

> If Linux is a city □, `systemd` is the mayor.

1.5.1.3 2) Understanding Units in systemd Everything in systemd is a **Unit**—a configuration file that tells systemd what to manage.

Type	Extension	Purpose
Service	`.service`	Background services
Timer	`.timer`	Scheduling
Mount	`.mount`	Mount points
Target	`.target`	Group of units

Type	Extension	Purpose
Socket	`.socket`	Socket activation

```
systemctl list-units           # List all active units
systemctl list-unit-files      # List all installed units
```

1.5.1.4 3) Managing Services with systemctl

This is daily Linux admin stuff.

```
sudo systemctl start apache2        # Start
sudo systemctl stop apache2         # Stop
sudo systemctl restart apache2      # Restart
sudo systemctl reload apache2       # Reload config (keeps connections
    alive)
sudo systemctl enable apache2       # Enable at boot
sudo systemctl disable apache2      # Disable at boot
sudo systemctl mask apache2         # Completely block from starting
sudo systemctl unmask apache2       # Unmask
systemctl status apache2            # Check status
```

Reload vs Restart (Important!): - `reload`: Reloads configuration, does NOT stop process, keeps connections alive - `restart`: Stops service, starts again, drops active connections

---

1.5.1.5 4) Where Are Unit Files Stored?

Directory	Purpose
`/usr/lib/systemd/system/`	Default package units
`/lib/systemd/system/`	(Debian equivalent)
`/etc/systemd/system/`	Custom & overrides

Priority Order (highest to lowest): `/etc/systemd/system/` overrides defaults.

1.5.1.6 5) Anatomy of a Service File

Inspect a service file structure:

```
systemctl cat apache2.service
```

Typical structure:

```
[Unit]
Description=...
After=network.target

[Service]
Type=forking
ExecStart=/usr/sbin/apachectl start
ExecReload=/usr/sbin/apachectl graceful
ExecStop=/usr/sbin/apachectl stop

[Install]
WantedBy=multi-user.target
```

[Unit] Section: Metadata and dependencies (Description, After=, Requires=)
[Service] Section: How the service runs (ExecStart, ExecStop, Restart, User, Type)
[Install] Section: Boot integration (`WantedBy=multi-user.target` means start in normal multi-user mode)

1.5.1.7 6) Creating Your Own Service Goal: Run a simple script at boot

Create script:

```
sudo nano /usr/local/bin/hello.sh
```

```
#!/bin/bash
echo "Hello from systemd!" >> /tmp/hello.log
```

Make executable:

```
sudo chmod +x /usr/local/bin/hello.sh
```

Create service file:

```
sudo nano /etc/systemd/system/hello.service
```

```
[Unit]
Description=My Hello Service

[Service]
```

```
Type=oneshot
ExecStart=/usr/local/bin/hello.sh

[Install]
WantedBy=multi-user.target
```

Reload systemd (IMPORTANT anytime you create/edit/delete unit files):

```
sudo systemctl daemon-reload
sudo systemctl start hello.service
cat /tmp/hello.log    # Boom \faIcon{exclamation-triangle}
```

1.5.1.8 7) Editing & Overriding Units (The Smart Way) Never edit files in `/lib/systemd/system/`.

Instead:

```
sudo systemctl edit apache2
```

This creates `/etc/systemd/system/apache2.service.d/override.conf`. Safe override.

```
[Service]
Restart=always
```

Save → `sudo systemctl daemon-reload` → restart service.

1.5.1.9 8) systemd Timers (Modern Scheduling) Now we level up. **Timers = replacement for cron.**

Why Use systemd Timers Instead of Cron? - Integrated with services - Better logging - More control - Dependency aware - Can use calendar expressions

> Note that systemd is more complex than Cron and requires systemd to work on your system.

Timer Structure: A timer requires (1) a `.service` file and (2) a `.timer` file.

1.5.1.10 9) Example: Send Message to All Users

Create script:

```
sudo nano /usr/local/bin/wall-msg.sh
```

```
#!/bin/bash
wall "System reminder: Stay awesome!"
```

Make executable:

```
sudo chmod +x /usr/local/bin/wall-msg.sh
```

Create service file:

```
sudo nano /etc/systemd/system/wall-msg.service
```

```
[Unit]
Description=Wall Message Service

[Service]
Type=oneshot
ExecStart=/usr/local/bin/wall-msg.sh
```

Create timer file:

```
sudo nano /etc/systemd/system/wall-msg.timer
```

```
[Unit]
Description=Run wall message every minute

[Timer]
OnCalendar=*-*-* *:*:00
Persistent=true

[Install]
WantedBy=timers.target
```

Breaking down the timer file:

`[Unit]` — The ID Card - `Description`: A human-readable label. It doesn't affect logic, but when you run `systemctl status`, this is what you see.

`[Timer]` — The Alarm Clock - `OnCalendar`: Defines *when* the task runs (explained in detail below). - `Persistent=true`: Acts like a "snooze" button for missed jobs. If your computer was turned off when the timer was supposed to trigger, systemd will **immediately run the task as soon as you turn the computer back on**.

`[Install]` — The Connection - `WantedBy=timers.target`: Tells systemd how to "hook" this timer into the system. When you run `systemctl enable`, it links this timer to the standard group of system timers so it starts automatically on boot.

Activate:

```
sudo systemctl daemon-reload
sudo systemctl enable wall-msg.timer
sudo systemctl start wall-msg.timer
systemctl list-timers    # Check timers \faIcon{exclamation-triangle}
```

1.5.1.11 10) Understanding OnCalendar `OnCalendar` uses a simple format: `DayOfWeek Year-Month-Day Hour:Minute:Second`

The Asterisk (`*`) means "every" or "don't care".

Expression	Meaning	Breakdown
`*-*-* *:*:00`	Every minute	Every year-month-day, every hour:minute, at second 00
`daily`	Once per day	Shortcut for `*-*-* 00:00:00`
`weekly`	Once per week	Shortcut for `Mon *-*-* 00:00:00`
`Mon *-*-* 09:00:00`	Every Monday at 9AM	Monday, any date, at 09:00:00
`*-*-01 00:00:00`	First day of every month	Any year-month, day 01, at midnight
`*-*-* *:00:00`	Every hour, on the hour	Any date, every hour, at minute 00, second 00
`Mon..Fri *-*-* 08:30:00`	Weekdays at 8:30AM	Monday through Friday, any date, at 08:30
`*-*-12,24 03:00:00`	12th and 24th of every month at 3AM	Any year-month, days 12 and 24, at 03:00

Pro Tips: - You can use shortcuts: `hourly`, `daily`, `weekly`, `monthly`, `yearly` - Use `systemd-analyze calendar "YOUR_EXPRESSION"` to test and verify your schedule before using it.

```
# Test a calendar expression
systemd-analyze calendar "Mon *-*-* 09:00:00"
```

This command will show you the next scheduled run time so you can verify your expression is correct.

1.5.1.12 11) Other Timer Options **Other Timer Options:** - `OnBootSec=` — After boot - `OnUnitActiveSec=` — After last run - `Persistent=true` — Run missed jobs after reboot

---

1.5.1.13 12) Timer vs Cron Comparison

Feature	Cron	systemd Timer
Logging	Basic	journalctl
Dependencies	No	Yes
Missed Runs	No	Yes (Persistent)
Integration	Separate	Native

Cron jobs for repetitive tasks:

```
crontab -l      # List cron jobs
crontab -e      # Edit cron jobs
```

1.5.1.14 13) Debugging Services & Timers Check service logs and failures:

```
journalctl -u wall-msg.service    # Logs for specific service
journalctl -f                     # Live logs
systemctl --failed                # Check failures
```

1.5.1.15 14) Real World Use Cases

- Backup scripts
- Log cleanup
- Health checks
- Auto-updates
- Dev server restarts
- IoT tasks

Common Mistakes: - ❌Forgetting `daemon-reload` - ❌Editing `/lib/systemd/system/` directly - ❌Wrong file permissions - ❌Forgetting `WantedBy=`

Final Mental Model: - Service = What to run - Timer = When to run - Unit file = Configuration blueprint - systemctl = Control center

> You are no longer a Linux user. You're becoming a Linux operator. →

1.5.1.16 Useful Links — Session 8 systemd Fundamentals:

🔗 Linux systemd and its components (GeeksforGeeks)

🔗 Introduction to systemd (GeeksforGeeks)

🔗 How to mask a systemd unit (GeeksforGeeks)

1.5.2 Session 9: Storage Management & Performance Optimization

> **`zram-tools` is optional but recommended for low-memory systems.**

1.5.2.1 Learning outcomes

- Inspect and mount storage devices correctly
- Configure persistent mounts through `/etc/fstab`
- Monitor memory behavior and choose swap/zram strategy

- Manage datasets/models under embedded storage constraints

1.5.2.2 1) Why This Session Matters

Before we touch commands, understand this: - Your **disk** stores data permanently - Your **RAM** is fast temporary memory - **Swap** is emergency backup memory - **zram** is compressed RAM swap (smart trick for low-memory systems)

If you understand this session, you'll mount extra drives safely, fix broken boot issues, add a second disk like a pro, understand RAM pressure, and boost low-RAM machines using zram.

This is not theory. This is real Linux power.

1.5.2.3 2) Understanding Your Disks `lsblk` — The Disk Tree Viewer:

```
lsblk
lsblk -f          # Shows filesystem, UUID, mountpoint (MORE DETAILED)
```

Example output:

```
sda├─
sda1├─
sda2└─
sda3
sdb└─
sdb1
```

Shows: Disk names, partitions, mount points, size, type.
⚠ Use this before and after plugging a USB.

`blkid` — UUID Finder:

```
sudo blkid
```

```
/dev/sda1: UUID="1A2B-3C4D" TYPE="vfat"
```

Why important? Because in `/etc/fstab`, we use **UUID**, not `/dev/sda1`. Why? Because device names can change on boot. **UUID never lies.**

1.5.2.4 3) Mounting and Unmounting Drives

Unlike Windows, Linux does not auto-use disks unless mounted.

Manual Mount:

```
sudo mkdir /mnt/mydrive           # Step 1: Create mount point
sudo mount /dev/sdb1 /mnt/mydrive # Step 2: Mount
ls /mnt/mydrive                   # Step 3: Check
```

Unmount:

```
sudo umount /mnt/mydrive
# OR
sudo umount /dev/sdb1
```

⚠ If busy error:

```
lsof | grep sdb1    # Find what's using it
```

1.5.2.5 4) Persistent Mounts with `/etc/fstab`

This file controls what mounts at boot.

```
sudo nano /etc/fstab
```

Example entry:

```
UUID=1a2b3c4d-xxxx-xxxx-xxxx-xxxxxxxxxxxx  /mnt/mydrive  ext4  defaults
      0  2
```

Column	Meaning
UUID	Disk identifier
Mount point	Where it appears
Filesystem	ext4, ntfs, vfat
Options	defaults, noatime
Dump	usually 0
fsck order	usually 2

[LAB] Test Before Reboot:

```
sudo mount -a    # If no errors → safe to reboot. If error → FIX IT
   before rebooting.
```

> **Pro tip:** Use `nofail` option in fstab for external drives so system doesn't panic if drive is missing.

1.5.2.6 5) Monitoring RAM Usage

RAM = short-term memory.

```
free -h
```

Example:

```
               total   used   free  shared  buff/cache  available
Mem:            7.6G   3.1G   1.2G     500M      3.3G       3.8G
Swap:           2.0G   0B     2.0G
```

Important columns: - **used** → currently used - **available** → real usable memory - **buff/cache** → Linux using RAM smartly

⚠ Linux uses RAM aggressively. That's GOOD. Unused RAM = wasted RAM.

Real-time view:

```
watch -n 1 free -h    # Updates every second
```

1.5.2.7 6) Understanding Swap

Swap = disk used as overflow memory. When RAM fills up → inactive pages move to swap. But swap is slower than RAM.

```
swapon --show         # Check swap
cat /proc/swaps
```

Creating a Swap File (Manual Way):

```
sudo fallocate -l 2G /swapfile    # Step 1
sudo chmod 600 /swapfile          # Step 2
sudo mkswap /swapfile             # Step 3
sudo swapon /swapfile             # Step 4
```

Make permanent—add to `/etc/fstab`:

```
/swapfile none swap sw 0 0
```

1.5.2.8 7) Swap and zram zram — Compressed RAM Swap:

zram creates compressed swap *in RAM*. Instead of writing to disk, it compresses pages and stores them in RAM—faster than disk swap.

Great for 4GB RAM laptops, old machines, and low-resource VPS.

Install on Debian/Ubuntu/Mint:

```
sudo apt install zram-tools
dpkg -l | grep zram              # Check if installed
systemctl status zramswap        # Check if active
```

Check config:

```
cat /etc/default/zramswap
```

You can adjust `PERCENT=70` (use 70% of RAM for zram—recommended: 70%-80%).

```
sudo systemctl restart zramswap
swapon --show                    # Should show /dev/zram0
```

Swap vs zram Comparison:

Feature	Disk Swap	zram
Speed	Slow	Faster
Uses disk	Yes	No
Best for	Large systems	Low RAM
Compression	No	Yes

> **Best practice for laptops:** Use zram + keep small disk swap.

1.5.2.9 8) Embedded Storage Choices (SD vs SSD)

- **SD:** Portable and common, but limited endurance
- **SSD:** Better durability/performance for heavy read/write workloads

---

1.5.2.10 9) Storage Lifespan Optimization Reduce unnecessary writes, rotate logs, and separate high-write data paths where possible.

1.5.2.11 10) AI and Simulation Storage Management Organize datasets/models/simulation outputs with clear cleanup and archival policy.

1.5.2.12 11) Troubleshooting Boot Failures (fstab Mistakes) If system won't boot: 1. Boot into recovery mode 2. Mount root 3. Edit `/etc/fstab` 4. Fix or comment broken line

---

1.5.2.13 12) Useful Extra Commands Additional disk and memory utilities:

```
df -h                 # Disk usage
du -sh *              # Folder sizes
mount | grep sdb      # See mount points
```

1.5.2.14 Practice Verify your disk and memory configuration:

```
lsblk
blkid
free -h
vmstat 1 5
df -h
```

1.5.2.15 Practical Lab Exercises

1. Plug USB → identify it using `lsblk`
2. Mount it manually in `/mnt/testusb`
3. Make it permanent in fstab
4. Create 1GB swap file
5. Install zram and compare swap usage

> **Final Advice:** Disk and memory management is not optional knowledge. It's what separates "Linux user" from "Linux operator". Break things in a VM. Test everything with `mount -a`. Never reboot blindly after editing fstab. You're building serious Linux skills now.

1.5.2.16 Useful Links — Session 9 Disk & Storage Tutorials:

- Adding 2nd drive tutorial (mmbesar)
- RAM, SWAP, ZRAM tutorial (mmbesar)
- Disk management video guide
- Swap & ZRAM video guide

1.5.3 Session 10: Shell Scripting & Robotics Networking

> **Verify packages:** `curl`, `aria2`, `ssh`, `net-tools`

1.5.3.1 Learning outcomes

- Build practical Bash scripts for robotics operations
- Diagnose and configure network connectivity
- Transfer data/models securely and efficiently
- Set up static IP and SSH keys for automation

1.5.4 Part 1: Bash Scripting for Robotics Automation

You already know how to navigate the terminal. You've used `grep`, `sed`, and `awk`. You've redirected output and written basic scripts. Now it's time to think like a developer and build tools that actually *do* things.

1.5.4.1 1) Variables and Data Types

In Bash, everything is a string. But that doesn't mean you can't be clever.

```bash
#!/bin/bash

# Variable assignment (no spaces!)
user_name="hacker"
age=21
is_cool=true

# Accessing variables
echo "Hey $user_name, you're $age years old!"

# Command substitution
current_date=$(date +%Y-%m-%d)
kernel_version=$(uname -r)

echo "Today is $current_date, running kernel $kernel_version"
```

> **Pro Tip:** Use `$()` for command substitution instead of backticks. It's nestable and easier to read.

Core script blocks to practice: Variables, Conditions and loops, Functions, Exit codes and basic error handling.

1.5.4.2 2) Control Flow If Statements:

```bash
#!/bin/bash

file_count=$(ls | wc -l)

if [ $file_count -gt 10 ]; then
    echo "Directory is cluttered ($file_count files). Time to clean up!"
elif [ $file_count -eq 0 ]; then
    echo "Empty directory. Lonely vibes."
else
    echo "Looking good with $file_count files."
fi
```

> **Important:** Always use spaces inside brackets `[ ]`. `[ $var -eq 1 ]` works, `[$var -eq 1]` fails.

Case Statements:

```bash
#!/bin/bash

echo "Choose your distro:"
echo "1) Arch"
echo "2) Debian"
echo "3) Fedora"
read choice

case $choice in
    1) echo "I use Arch, btw" ;;
    2) echo "Stable and reliable. Nice." ;;
    3) echo "Red Hat family represent!" ;;
    *) echo "Invalid choice. Try again." ;;
esac
```

1.5.4.3 3) Loops **For Loops:**

```bash
#!/bin/bash

for file in *.txt; do
    echo "Processing $file..."
    wc -l "$file"
done

# C-style
for ((i=0; i<5; i++)); do
    echo "Iteration $i"
done
```

While Loops:

```bash
#!/bin/bash

# Read file line by line
while IFS= read -r line; do
    echo "Line: $line"
done < input.txt

# With break
counter=0
while true; do
    echo "Loop $counter"
    ((counter++))
    [ $counter -eq 5 ] && break
done
```

1.5.4.4 4) Functions

Write reusable code blocks:

```bash
#!/bin/bash

check_port() {
    local host=$1
    local port=$2
    timeout 2 bash -c "</dev/tcp/$host/$port" 2>/dev/null && \
        echo "Port $port on $host is OPEN" || \
        echo "Port $port on $host is CLOSED"
}

check_port "google.com" 443
check_port "localhost" 22
```

Key Points: - Use `local` for variables inside functions to avoid global scope pollution - Functions should do one thing well (Unix philosophy) - Return values are 0-255 (exit codes), not strings

1.5.4.5 5) Error Handling

Don't let your scripts crash silently:

```bash
#!/bin/bash

# Automatic error handling
set -euo pipefail
# -e: Exit on error
# -u: Exit on undefined variable
# -o pipefail: Catch errors in pipelines

# Or handle manually
if ! ping -c 1 google.com &> /dev/null; then
    echo "ERROR: No internet connection" >&2
    exit 1
fi
```

1.5.4.6 6) Working with APIs

Modern scripting is all about data:

```bash
#!/bin/bash

response=$(curl -s "https://api.github.com/users/torvalds")
```

```
login=$(echo "$response" | jq -r '.login')
followers=$(echo "$response" | jq -r '.followers')
created_at=$(echo "$response" | jq -r '.created_at')

echo "User: $login"
echo "Followers: $followers"
echo "Joined: $created_at"
```

1.5.5 Part 2: Networking Essentials

Linux networking tools are your eyes and ears into the digital world. Whether you're debugging connectivity, downloading files, or managing remote systems, these commands are essential.

1.5.5.1 1) Connectivity Testing ping — The Classic:

```
ping google.com
ping -c 4 8.8.8.8           # Stop after 4 packets
sudo ping -i 0.2 google.com # Faster ping
ping -a 192.168.1.1         # Audible ping
```

traceroute / tracepath:

```
traceroute google.com       # See the path your packets take
tracepath google.com        # Alternative without root
```

1.5.5.2 2) Network Interface Configuration ip (Modern Replacement for ifconfig):

```
ip a                        # Show all interfaces (short form)
ip addr show eth0           # Specific interface
sudo ip link set eth0 up    # Bring interface up/down
sudo ip link set eth0 down
sudo ip addr add 192.168.1.100/24 dev eth0  # Add IP
ip route show               # Routing table
sudo ip route add default via 192.168.1.1   # Add route
```

Why `ip` over `ifconfig`? `ip` is part of the iproute2 package, actively maintained, and supports modern networking features.

ifconfig (Legacy—still works on many systems):

```
ifconfig
ifconfig eth0
```

1.5.5.3 3) Data Transfer curl — The URL Swiss Army Knife:

```
curl https://api.example.com/data                # GET request
curl -o file.zip https://example.com/file.zip    # Save to file
curl -L https://bit.ly/xyz                       # Follow redirects
curl -I https://google.com                       # Headers only
curl -X POST -d "name=john" https://api.example.com/users  # POST
curl -C - -o large.iso https://example.com/large.iso       # Resume
    download
curl -s https://api.ipify.org                    # Silent mode
curl -I https://example.com                      # Check headers
```

wget — The Download Specialist:

```
wget https://example.com/file.zip                # Simple download
wget -P ~/Downloads https://example.com/file.zip # To specific dir
wget -c https://example.com/large.iso            # Resume
wget -b https://example.com/large-file.tar.gz    # Background
wget --limit-rate=500k https://example.com/file.zip # Limit speed
wget -r -A.pdf https://example.com/documents/    # Recursive download
```

aria2 — The Download Accelerator:

```
aria2c -x 16 -s 16 https://example.com/large.iso # Multi-connection (
    faster!)
aria2c -c https://example.com/large.iso           # Resume
aria2c --max-download-limit=2M https://example.com/file.zip
```

> **curl vs wget:** Use `curl` for APIs and complex HTTP. Use `wget` for reliable downloads and recursive fetching.

1.5.5.4 4) Remote Access ssh — Secure Shell:

```
ssh username@remote-host                         # Basic connection
ssh -p 2222 username@remote-host                 # Specific port
ssh user@host "ls -la /var/log"                  # Execute command
    remotely
ssh -L 8080:localhost:80 user@host               # Tunnel
```

```
ssh -D 1080 user@host                          # SOCKS proxy
ssh -X user@host                               # X11 forwarding
ssh-copy-id user@host                          # Key-based auth (no
    password!)
ssh -i ~/.ssh/my_key user@host                 # Use specific key
ssh user@robot-host
```

SSH Key Generation:

```
ssh-keygen -t ed25519
```

> **Next Steps:** Disable password authentication in `/etc/ssh/sshd_config`. Use key pairs with passphrase. Change default port. Use `fail2ban` to block brute force attempts.

scp — Secure Copy:

```
scp file.txt user@host:/home/user/             # Local to remote
scp user@host:/var/log/syslog ./local-syslog   # Remote to local
scp -r ./project user@host:/var/www/           # Directory
scp -p file.txt user@host:/backup/             # Preserve attributes
scp model.bin user@robot-host:/tmp/
```

rsync — Efficient Incremental Sync:

```
rsync -av /src/ /dest/                         # Local sync
rsync -avz /src/ user@host:/dest/              # Remote sync
rsync -av logs/ user@robot-host:/tmp/logs/
```

> Learn the difference trailing slash makes—it's the "slash of power" vs.

1.5.5.5 5) Multi-Robot Network Configuration Plan IP ranges, naming conventions, and role-based segmentation.

1.5.5.6 6) Static IP Setup for Raspberry Pi Robots Stable addressing is critical for predictable remote control and orchestration.

1.5.5.7 7) SSH Key-Based Authentication Use keys for secure, passwordless automation across robot fleets.

1.5.5.8 Useful Links — Session 10 Bash Scripting:

- Bash scripting tutorial (YouTube)
- BashGuide
- ShellCheck — lint your scripts
- ExplainShell — understand complex commands

Networking Tools:

- curl documentation
- GNU Wget Manual
- aria2 manual
- OpenSSH Manual

Man Pages:

- ping
- ip
- ifconfig
- scp

1.5.6 Session 11: Building Embedded Linux Systems with Buildroot

> **Requirements:** 20 GB+ free storage, stable internet
> **Tutorial reference:**

1.5.6.1 Learning outcomes

- Explain Buildroot vs Yocto tradeoffs
- Configure and build a minimal embedded Linux image
- Interpret output artifacts (kernel/rootfs/bootloader)
- Apply basic customization and package integration

1.5.6.2 1) Build Systems Overview **Buildroot:** simpler, faster, approachable for learning and many practical projects
Yocto: highly flexible and powerful, but steeper complexity

1.5.6.3 2) Buildroot Architecture and Workflow Pipeline: 1. Get source 2. Select base configuration 3. Configure options 4. Build toolchain + packages + images 5. Collect final artifacts

---

1.5.6.4 3) Configuration Basics Start with a baseline, then customize:

```
make defconfig          # Baseline setup
make menuconfig         # Interactive customization
```

Core choices: target architecture, toolchain, init system, filesystem image format.

1.5.6.5 4) First Build and Outputs Run the build and inspect the output:

```
make                    # Build everything
ls output/images        # Check results
```

`make` produces images such as: - Root filesystem - Kernel image - Bootloader-related artifacts

---

1.5.6.6 5) Customization

- Add packages through menuconfig
- Use rootfs overlays for custom files
- Add post-build scripts
- Create custom packages with `.mk` and `Config.in`

1.5.6.7 6) Practical Exercise Build a minimal ARM system for Raspberry Pi emulation including Python and SSH support.

```
make defconfig
make menuconfig
make
ls output/images
```

1.5.6.8 Useful Links — Session 11 Buildroot:

Buildroot tutorial video

Arch Wiki:

Arch Wiki — the ultimate Linux reference

1.5.6.9 Useful Links — Cool Stuff & Extras Multimedia:

FFmpeg tutorial (It's FOSS)

FFmpeg docs

FOSS Alternatives:

Top 10 FOSS Apps (Neowin)

AlternativeTo.net

Windows Apps/Games on Linux:

🔗 Bottles + Wine (YouTube)

🔗 Lutris (YouTube)

Sync & Backup:

🔗 DigitalOcean rsync tutorial

🔗 rsync backup tutorial (YouTube)

Ricing & Customization:

🔗 Ricing inspiration (YouTube)

🔗 Perfect Ubuntu Guide (GitHub)

🔗 Best Arabic Font in Linux (mmbesar)

🔗 Linux Utilities (GitHub Tinram)

🔗 12 GREAT CLI programs (YouTube)

Shell & Scripting Resources:

🔗 Shell scripting course (Learn By Example)

1.6 Suggested Projects

1. **System Monitor Script:** Create a Bash monitor script for CPU/RAM with logging and >80% alerts.
2. **Automated Backup:** Build an automated workspace backup script to remote/cloud storage.
3. **SSH Remote Execution:** Practice SSH remote command execution between two machines/VMs (or localhost).
4. **QEMU Emulation:** Run full Raspberry Pi OS emulation in QEMU.
5. **Mobile SSH:** Optional: manage your Linux machine remotely from mobile (Termux + SSH).
6. **Custom Buildroot Image:** Build and document a custom Buildroot image with Python + SSH.
7. **Info Script:** Write a bash script that fetches currency rates, prayer times, or weather forecast using APIs with colored output and error handling.
8. **Rice Your Desktop:** Create a customized desktop environment with documented dotfiles on Git.

DONE !! thanks Allah.